

ARRAYS AND IS METHODS

ex 1:

Create a JavaScript program to manage a list of products for an e-commerce site. The program should:

Add new products.

1.Remove products by ID.

2.Update product details by ID.

3.Calculate the total inventory value.

4.Find products within a specified price range.

5.Sort products by price.

6.Search for products by name.

Instructions:

7.Initialize the Array:

Create an array named products where each product is an object with id, name, and price properties. For example:

Add New Products:

Write a function addProduct that takes a product object as an argument and adds it to the products array.

Remove Product by ID:

Write a function removeProductById that takes a product ID as an argument and removes the product with that ID from the products array.

Update Product Details by ID:

Write a function updateProductById that takes a product ID and an object containing the updated details (name and/or price) as arguments and updates the corresponding product in the products array.

Calculate Total Inventory Value:

Write a function calculateTotalValue that calculates and returns the total value of all products in the products array by summing up their prices.

Find Products by Price Range:

Write a function findProductsInRange that takes a minimum and maximum price and returns a new array containing products whose prices fall within that range.

Sort Products by Price:

Write a function sortProductsByPrice that sorts the products array in ascending order based on product prices.

Search Products by Name:

Write a function `searchProductsByName` that takes a search term and returns an array of products whose names include the search term.

Solution:

```
let products = [
  { id: 1, name: 'Laptop', price: 999 },
  { id: 2, name: 'Smartphone', price: 699 },
  { id: 3, name: 'Tablet', price: 499 }
];

// Add a new product
function addProduct(product) {
  products.push(product);
}

// Remove a product by ID
function removeProductById(id) {
  products = products.filter(product => product.id !== id);
}

// Update product details by ID
function updateProductById(id, updatedDetails) {
  products = products.map(product => {
    if (product.id === id) {
      return { ...product, ...updatedDetails };
    }
    return product;
  });
}

// Calculate total inventory value
function calculateTotalValue() {
  return products.reduce((acc, product) => acc + product.price, 0);
}

// Find products within a specified price range
function findProductsInRange(minPrice, maxPrice) {
  return products.filter(product => product.price >= minPrice && product.price <= maxPrice);
}

// Sort products by price
function sortProductsByPrice() {
  products.sort((a, b) => a.price - b.price);
}

// Search products by name
function searchProductsByName(searchTerm) {
  return products.filter(product => product.name.toLowerCase().includes(searchTerm.toLowerCase()));
}

// Test the functions
addProduct({ id: 4, name: 'Headphones', price: 199 });
console.log('Products after adding new product:', products);

removeProductById(2);
console.log('Products after removing ID 2:', products);

updateProductById(3, { price: 450 });
console.log('Products after updating ID 3:', products);

console.log('Total inventory value:', calculateTotalValue());

console.log('Products in the range $400 - $1000:', findProductsInRange(400, 1000));

sortProductsByPrice();
console.log('Products after sorting by price:', products);

console.log('Products containing "phone":', searchProductsByName('phone'));
```